3 Ranking with Indexes

3.1 Overview

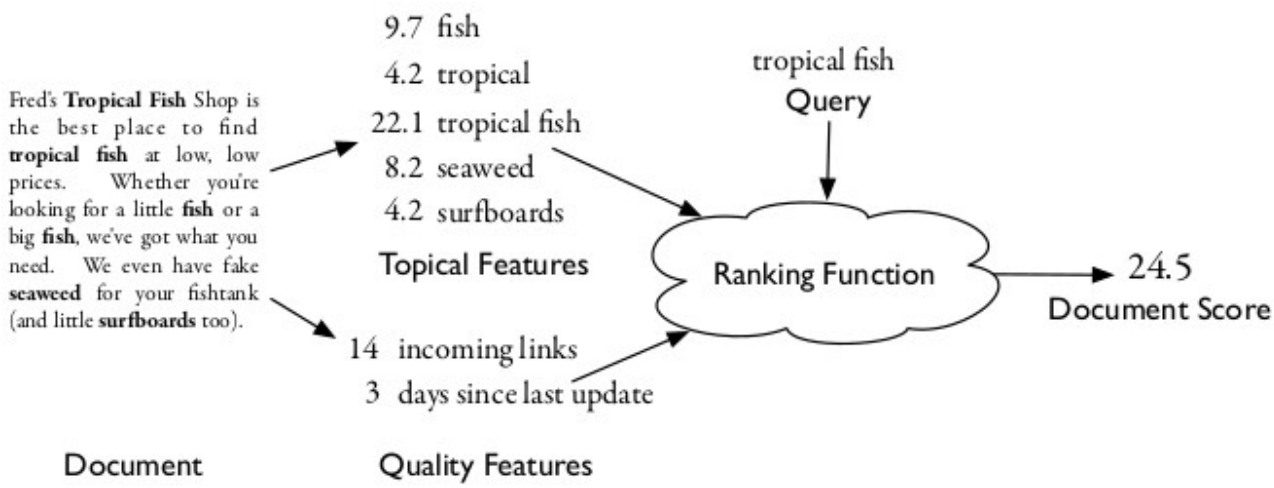
- Indexes are data structures designed to make search faster
- Most common data structure is inverted index
  - general name for a class of structures
  - “inverted” because documents are associated with words, rather than words with documents

3.2 Models of Ranking

3.2.1 Indexing and Ranking

- Indexes are designed to support search
  - faster response time, supports updates
- Text search engines use a particular form of search: ranking
  - documents are retrieved in sorted order according to a score computing using the document representation, the query, and a ranking algorithm

3.2.2 An abstract model for ranking

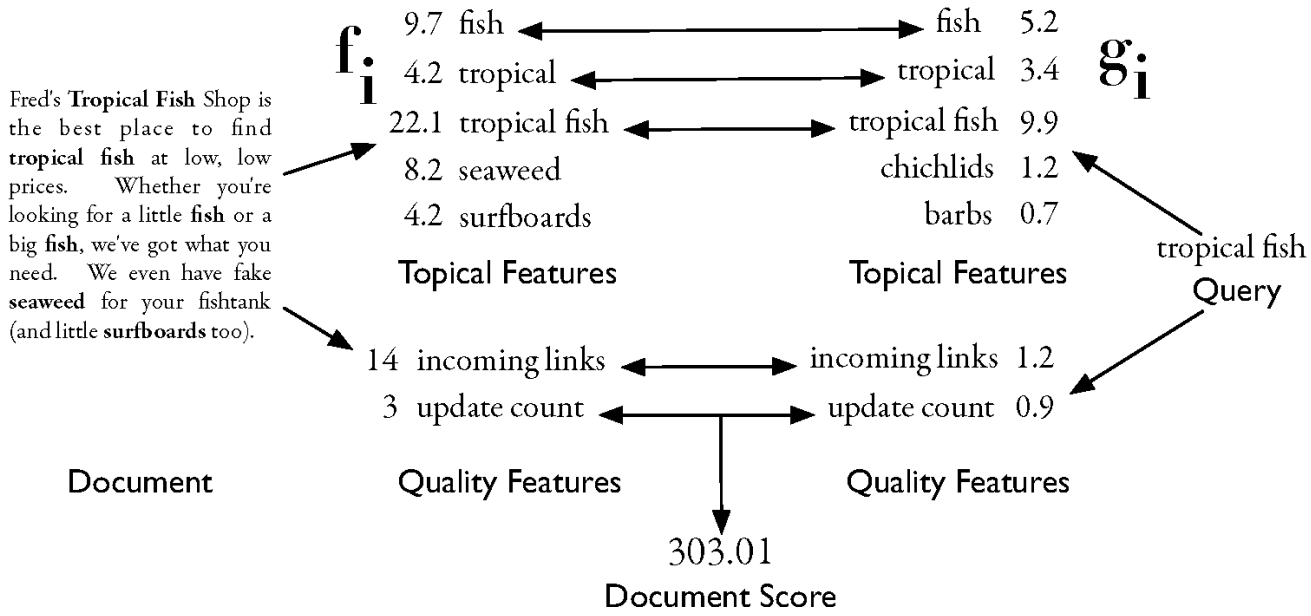


- Left: a sample document

  Document text is transformed into index terms (i.e., document features)

  Two kinds of features in the diagram
- Top:
  - Topical features
    - A number that estimates the degree to which the document is about a particular subject
- Bottom
  - Shows two possible document quality features
    - Number of web pages that link to this document
    - Number of days since this page was last updated
  - These features don't address whether the document is a good topical match for a query, but they do address its quality
    - A page with no incoming links that hasn't been edited in years is probably a poor match for any query
    - Some mathematical expression can be formulated to values for these features
- Right
  - Ranking function
    - Combines data from document features with the query to produce a score
      - High score: the system thinks that document is a good match for the query
      - Low score: the system thinks the document is a poor match for the query
- To build a ranked list of results, the documents are sorted by score so that the highest-scoring documents come first.
- Most reasonable ranking functions ignore many of the document features, and focus only on the small subset that relate to the query

3.2.3 A more concrete model for ranking



$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

f_i is a document feature function
g_i is a query feature function

Features

Document features

- Topical features
  - These correspond to the $f_i(D)$ functions in the equation for ranking
    - Examples: $f_{tropical}(D)$ or $f_{fish}(D)$
    - These values will be larger for documents that contain the words “tropical” or “fish” more often or more prominently
- Non-topical features
  - E.g., the fact that this document
    - has been updated three times
    - has 14 incoming links
  - These are hints about the quality of the document, e.g.,

        - The document wasn't just posted to the Web and then abandoned, since it gets updated occasionally
        - There are 14 other pages that have links pointing to it, which might mean that it has some useful information on it

Query features examples

- Topical features
    - Feature function $g_{tropical}(Q)$ evaluates to a large value because "tropical" (which matches a feature in the document) is in the query
    - However, $g_{barbs}(Q)$ also has a small non-zero value because it is related to other terms (viz, fish) in the query
- Non-topical query features
    - E.g., the update count feature
        - This doesn't mean that the query has been updated
        - The value of this feature indicates how important document updates are to relevance for this query
        - Example: For the query "today's weather in london", we would prefer documents that are updated frequently, since a document that isn't updated at least daily is unlikely to say anything interesting about today's weather
            - Hence, this query should have a high value for the update count feature
        - By contrast, a document that never changed could be very relevant for the query "full text of Shakespeare's Merchant of Venice"
            - This query could have a low feature value for update count

Final document score

- Multiply values from the query feature functions by values from document feature functions, then summed

Efficiency consideration

- If a retrieval system had to perform a sum over millions of features for every document, text search systems would not be practical
- In practice
    - a retrieval system does not perform a sum over millions of features for every document
        - query features (gi (Q)) are mostly zero
    - Hence, the sum for each document is only over the non-zero $g_i(Q)$ values

3.3 Inverted Indexes

3.3.1 Recall

- Each index term is associated with an inverted list
- Contains lists of documents, or lists of word occurrences in documents, and other information
- Each entry is called a posting
- Each document in the collection is given a unique number
- Lists are usually document-ordered (sorted by document number)

3.3.2 Documents

Example collection

S_1 Tropical fish include fish found in tropical environments around the world, including both freshwater and salt water species.

S_2 Fishkeepers often use the term tropical fish to refer only those requiring fresh water, with saltwater tropical fish referred to as marine fish.

S_3 Tropical fish are popular aquarium fish, due to their often bright coloration.

S_4 In freshwater fish, this coloration typically derives from iridescence, while salt water fish are generally pigmented.

3.3.2.1 Simple inverted index

and	1			
aquarium	3			
are	3	4		
around	1			
as	2			
both	1			
bright	3			
coloration	3	4		
derives	4			
due	3			
environments	1			
fish	1	2	3	4
fishkeepers	2			
found	1			
fresh	2			
freshwater	1	4		
from	4			
generally	4			
in	1	4		
include	1			
including	1			
iridescence	4			
marine	2			
often	2	3		
only	2			
pigmented	4			
popular	3			
refer	2			
referred	2			
requiring	2			
salt	1	4		
saltwater	2			
species	1			
term	2			
the	1	2		
their	3			
this	4			
those	2			
to	2	3		
tropical	1	2	3	
typically	4			
use	2			
water	1	2	4	
while	4			
with	2			
world	1			

3.3.2.2 *Inverted index with counts*

- Supports better ranking algorithms

and	1:1			
aquarium	3:1			
are	3:1	4:1		
around	1:1			
as	2:1			
both	1:1			
bright	3:1			
coloration	3:1	4:1		
derives	4:1			
due	3:1			
environments	1:1			
fish	1:2	2:3	3:2	4:2
fishkeepers	2:1			
found	1:1			
fresh	2:1			
freshwater	1:1	4:1		
from	4:1			
generally	4:1			
in	1:1	4:1		
include	1:1			
including	1:1			
iridescence	4:1			
marine	2:1			
often	2:1	3:1		

only	2:1		
pigmented	4:1		
popular	3:1		
refer	2:1		
referred	2:1		
requiring	2:1		
salt	1:1	4:1	
saltwater	2:1		
species	1:1		
term	2:1		
the	1:1	2:1	
their	3:1		
this	4:1		
those	2:1		
to	2:2	3:1	
tropical	1:2	2:2	3:1
typically	4:1		
use	2:1		
water	1:1	2:1	4:1
while	4:1		
with	2:1		
world	1:1		

3.3.2.3 *Inverted index with word positions*

- Supports proximity matches

and	1,15				
aquarium	3,5				
are	3,3	4,14			
around	1,9				
as	2,21				
both	1,13				
bright	3,11				
coloration	3,12	4,5			
derives	4,7				
due	3,7				
environments	1,8				
fish	1,2	1,4	2,7	2,18	2,23
			3,2	3,6	4,3
			4,13		
fishkeepers	2,1				
found	1,5				
fresh	2,13				
freshwater	1,14	4,2			
from	4,8				
generally	4,15				
in	1,6	4,1			
include	1,3				
including	1,12				
iridescence	4,9				

marine	2,22				
often	2,2	3,10			
only	2,10				
pigmented	4,16				
popular	3,4				
refer	2,9				
referred	2,19				
requiring	2,12				
salt	1,16	4,11			
saltwater	2,16				
species	1,18				
term	2,5				
the	1,10	2,4			
their	3,9				
this	4,4				
those	2,11				
to	2,8	2,20	3,8		
tropical	1,1	1,7	2,6	2,17	3,1
typically	4,6				
use	2,3				
water	1,17	2,14	4,12		
while	4,10				
with	2,15				
world	1,11				

3.3.2.4 *Proximity Matches*

- Matching phrases or words within a window
  - e.g., "tropical fish", or “find tropical within 5 words of fish”
- Word positions in inverted lists make these types of query features efficient
- e.g., Aligning posting lists for “tropical” and “fish” to find the phrase “tropical fish”

tropical	1,1		1,7	2,6	2,17		3,1			
fish	1,2	1,4		2,7	2,18	2,23	3,2	3,6	4,3	4,13

3.3.2.5 Fields and Extents

Document structure is useful in search

- Field, e.g., **date**, **from**:, etc. in emails
- Some fields more important
  - e.g., title

Options

- Create separate inverted lists for each field type
- Add information about fields to postings
- Use extent lists

- Creating separate inverted lists for each field type
  - e.g., one for document titles, one for document headings, and one for body text
  - Searching for words in the title is as simple as searching the title index
  - However, finding a word in any section of the document is trickier, since you need to fetch inverted lists from many different indexes to make that determination
- Adding information about fields to postings
  - Example, we could specify that the number 0 indicates a title and 1 indicates body text.
  - Each inverted list posting would then contain a 0 or a 1 at the end
  - This data could be used to quickly determine whether a posting was in a title, and it would require only one bit per posting
  - However, if you have more fields than just a title, the representation will grow.
- Using extent lists
  - The two suggestions have problems when faced with more complicated kinds of document structure
  - Example
    - Suppose we want to index books
    - Some books have more than one author
    - Sample XML description of a book with three authors

      <author>W. Bruce Croft</author>,

      <author>Donald Metzler</author>, and

      <author>Trevor Strohman</author>

- Suppose you would like to find books by an author named Croft Donald. Should the query ”croft donald” result to a match with the book?
  - Probably not. Even though the words “croft” and “donald” appear in it, they are in two distinct author fields
  - The previous two methods for dealing with fields (bits in the posting list, separate indexes) cannot make this kind of distinction
  - This is where extent lists come in

Extent Lists

- An extent is a contiguous region of a document
  - Can be represented using word positions
  - E.g.: if the title of a book started on the fifth word and ended just before the ninth word, we could encode that as (5,9)
  - For the author text above, we could write
    - author: (1,4), (4,6), (7,9)
      - (1,4) means that the first three words (“W. Bruce Croft”), i.e., first author, followed by the second author
      - (4, 6) (“Donald Metzler”), which is two words
      - The word “and” is not in an author field, but the next two words are, so the last posting is (7,9)
  - Inverted list records all extents for a given field type
    - e.g., Aligning posting lists for “fish” and title to find matches of the word “fish” in the title field of a document

fish	1,2	1,4		2,7	2,18	2,23	3,2	3,6	4,3	4,13
title	1:(1,3)		2:(1,5)							4:(9,15)

      - Diagram shows the same positions posting list for “fish” that we used earlier
      - We also have an extent list for the title field (assuming our documents have titles)
        - At the very beginning of both lists, we see that document 1 has a title that contains the first two words (1 and 2, ending just before the third word)
        - We know that this title includes the word “fish”, because the inverted list for “fish” tells us that “fish” is the second word in document 1
      - If the user wants to find documents with the word “fish” in the title
        - Document 1 is a match

        - Document 2 does not match, because its title ends just before the fifth word, but “fish” doesn’t appear until the seventh word
        - Document 3 apparently has no title at all, so no matches are possible
        - Document 4 has a title that starts at the ninth word (perhaps the document begins with a date or an author declaration), and it does contain the word “fish”
        - In all, this example shows two matching documents: 1 and 4
- Extent lists contd.
    - Concept can be extended to all kinds of fields: headings, paragraphs, or sentences
    - It can also be used to identify smaller pieces of text with specific meaning, such as addresses or names

3.3.2.6 Scores

- If the inverted lists are going to be used as the basis of computing values from inverted list of features functions, why not just store the value of the feature function?
    - Precomputed scores in inverted list
    - We could make a list for “**fish**” that has postings like [(1:3.6), (3:2.2)]
        - 3.6 is total feature value for document 1
        - 2.2 is total feature value for document 3
        - Feature score is some indicator of how much this document is about fish
    - Some possible sources of total feature score
        - Number of times “fish” appeared in the title, in the headings, in large fonts, in bold, and in links to the document
        - Maybe the document doesn’t contain the word “fish” at all, but instead many names of fish, such as “carp” or “trout”
- Precomputed scores in inverted list
    - improves speed as much of the hard work in computing scores is moved to the offline indexing process
    - but reduces flexibility as scores are pre-computed, and we can no longer change the scoring mechanism once the index is built

3.3.2.7 Ordering

- Score-ordered lists
    - Sometimes, postings of each inverted list are not ordered by document number
        - Can also be ordered by score, so that the highest-scoring documents come first

- Makes sense only when the lists already store the score, or when only one kind of score is likely to be computed from the inverted list
- By storing scores instead of documents, the query processing engine can focus only on the top part of each inverted list, and only read the first k postings
  - In a traditional document-ordered inverted list, the query processing engine would need to scan the whole list to find the top k scoring documents

3.4 Compression

- Inverted lists are very large
- Compression of indexes saves disk and/or memory space
  - Typically have to decompress lists to use them
  - Best compression techniques have good compression ratios and are easy to decompress
- Lossless compression – no information lost
- See text for various approaches to compression

3.5 Auxiliary Structures

- Searching the index
  - Some kind of data structure is necessary to find the inverted list for a particular term
  - Simplest (but costly) approach
    - Store each inverted list as a separate file (name each file with the search term)
    - However, document collections can have millions of unique words, most of them occurring only once or twice in the collection
    - Millions of very small files containing these terms
    - Disadvantages of doing this
      - File systems are not optimized for this kind of storage (typically, a few kilobytes of space used up, even though most files will contain just a few bytes of data
      - Result: huge amount of wasted space
      - Another problem: too many files mean too many (slow) disk accesses
    - To fix these problems, inverted lists are usually stored together in a single file (the inverted file)
      - An additional directory structure (the vocabulary or lexicon) contains a lookup table from index terms to the byte offset of the inverted list in the inverted file
      - In many cases, this vocabulary lookup table will be small enough to fit into memory

        - The vocabulary data can be stored in any reasonable way on disk and loaded into a hash table at search engine startup
        - If the search engine needs to handle larger vocabularies, some kind of tree-based data structure, such as a B-tree, should be used to minimize disk accesses during the search process
    - To compute some feature functions, the index needs to contain certain vocabulary statistics (term frequency, document frequency)
    - When these statistics pertain to a specific term, they can be easily stored at the start of the inverted list
    - Collection statistics stored in separate file

3.6 Index Construction

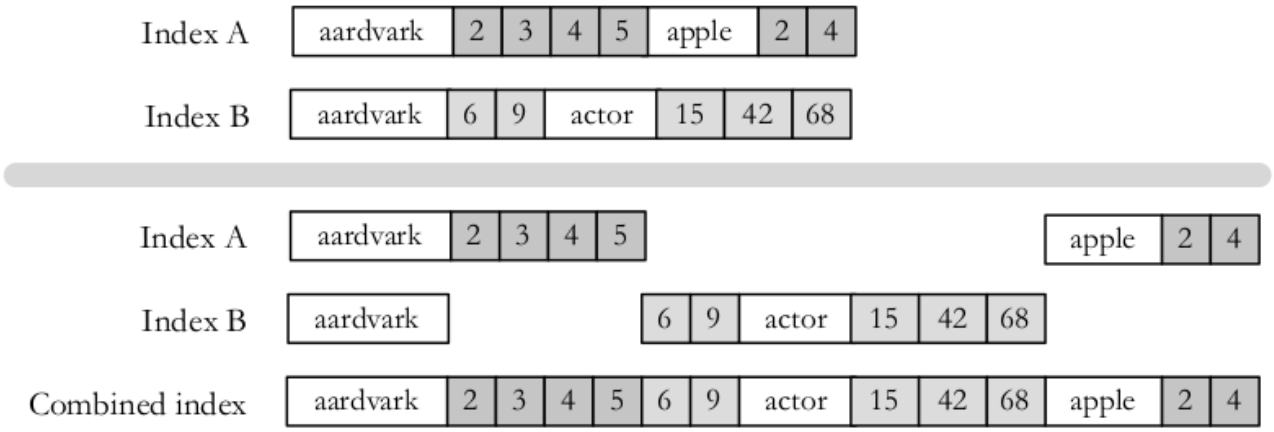
- Index is created from the text collection
- Building a small index is not particularly difficult, but for large input sizes, we need index construction tricks
- We consider
    - Simple in-memory index construction
    - Case where the input data does not fit in memory
    - Building indexes using more than one computer

3.6.1 Simple index construction

- Requires that all of the inverted lists be stored in memory
    - Pass list of documents is passed to the BuildIndex function, which parses each document into tokens
        - i.e., words, perhaps with some additional processing, such as downcasing or stemming
    - Remove duplicate tokens (perhaps, using a hash table)
    - For each token, determine whether a new inverted list needs to be created, and create one if necessary
    - Add document number (ID), n, to the inverted list
- The result is a hash table of tokens and inverted lists
    - The inverted lists are just lists of integer document numbers and contain no special information (although we could store term statistics as well)
    - Okay for many small tasks, e.g., indexing less than a few thousand documents

3.6.2 Merging

- This is the classic way to solve the memory problem in the previous example
- Build the inverted list structure ***I*** until memory runs out
- When that happens, write the partial index ***I*** to disk, then start making a new one
- We end up with with many partial indexes, I1 , I2 , I3 , ..., In on disk
- The system then merges these files into a single result
- Notes
  - Memory cannot hold even two of the partial index files at one time, so the input files need to be carefully designed so that they can be merged in small pieces
  - One approach
    - Store the partial indexes in alphabetical order
    - Use a merge algorithm to merge the partial indexes using very little memory



- Note
  - This merging strategy also shows a possible parallel indexing strategy
    - If many machines build their own partial indexes, a single machine can combine all of those indexes together into a single, final index

3.6.3 Distributed Indexing

- Distributed processing driven by need to index and analyze huge amounts of data (e.g., the Web)
- Large numbers of inexpensive servers used rather than larger, more expensive machines
- MapReduce is a distributed programming tool designed for indexing and analysis tasks

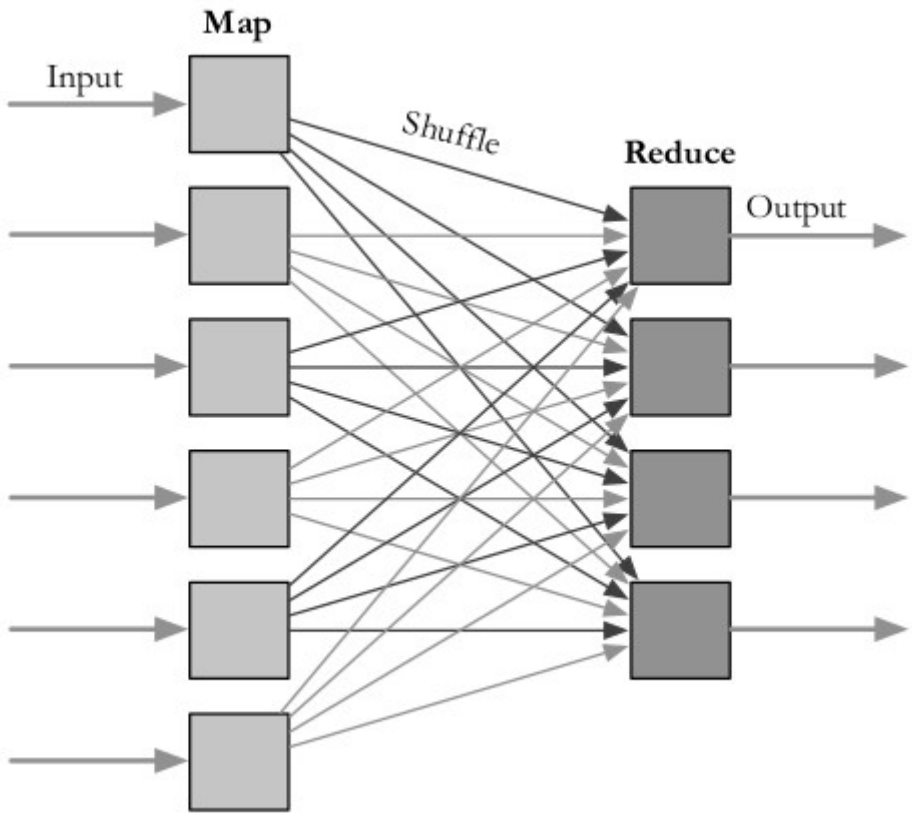
Motivating example

- Given a large text file that contains data about credit card transactions
  - Each line of the file contains a credit card number and an amount of money
  - We want to determine the number of unique credit card numbers
- Some possibilities
  - (1)
    - If the file is not very big, you could read each line, parse the credit card number, and store the credit card number in a hash table
    - Once the entire file had been read, the hash table would contain one entry for each unique credit card number
    - Counting the number of entries in the hash table would give you the answer
    - Not feasible for a big file, as the hash table would be too large to store in memory
  - (2)
    - Assume that the transactions in the file were ordered by credit card number
    - Counting the number of unique credit card numbers in this case is very simple
    - Simply read each line in the file and parse the credit card number on the line
    - If the credit card number found is different from the one on the line before it, a counter is incremented
    - At the end of the file, the counter contains a count of the unique credit card numbers in the file. No hash table is necessary for this to work.
  - (3)
    - Distributed computation. Suppose you have more than one computer to use for this counting task
    - Can split the big file of transactions into small batches of transactions
    - Each computer can count its fraction, and then the results can be merged together to produce a final result
    - **Scenario 1 (start with an unordered file of transactions)**
      - Split that file into small batches of transactions and count the unique credit card numbers in each batch
      - Combine the results by adding the number of credit card numbers found in each batch, but this is incorrect, since the same credit card number might appear in more than one batch, and therefore would be counted more than once in the final total
      - Instead, we would need to keep a list of the unique credit card numbers found in each batch, and then merge those lists together to make a final result list. The size of this final list is the number of unique credit card numbers in the whole set

        - Scenario 2 (Split the transactions into batches, taking care to ensure that all transactions made with the same credit card end up in the same batch
            - Each batch can now be counted individually, and then the counts from each batch can be added to make a final result. No merge is necessary, as each credit card number will appear in precisely one batch
- The big picture
    - Proper data grouping can radically change the performance of a task
    - Sorted input file made the counting task easy, reduced the amount of memory needed to nearly zero, and made it possible to distribute the computation easily

3.6.4 MapReduce

- A distributed programming framework that focuses on data placement and distribution
- Comprises
    - Mapper
        - Generally, transforms a list of items into another list of items of the same length
    - Reducer
        - Transforms a list of items into a single item
- The MapReduce framework isn't quite so strict with its definitions
    - Both Mappers and Reducers can return an arbitrary number of items. However, the general idea is the same
- The MapReduce library automatically launches many Mapper and Reducer tasks on a cluster of machines
- MapReduce Basic Process
    - Map stage
        - We assume that the data comes in a set of records. The records are sent to the
        - Mapper transforms data records into pairs, each with a key and a value
    - Shuffle uses a hash function so that all pairs with the same key end up next to each other and on the same machine
    - Reduce stage processes records in batches, where all pairs with the same key are processed at the same time
- Idempotence of Mapper and Reducer provides fault tolerance
    - Multiple operations on same input gives same output



3.6.5 Updates

- Most document collections are constantly changing
  - Collections get bigger over time
  - The contents of documents can change over time
- Search engine needs to be able to respond to dynamic collections
- Two techniques to solve the problem of update: index merging and result merging
- Index merging
  - Typically, index is stored on a disk
  - Inserting data in the middle of a file is not supported by any common file system, so direct disk-based update is not straightforward
  - The merging approach we saw in 3.6.2. can be use for adding data to the index
    - Make a new, smaller index (I2) and merge it with the old index (I1) to make a new index containing all of the data (I)
    - Postings in I1 for any deleted documents can be ignored during the merge phase so they do not appear in I

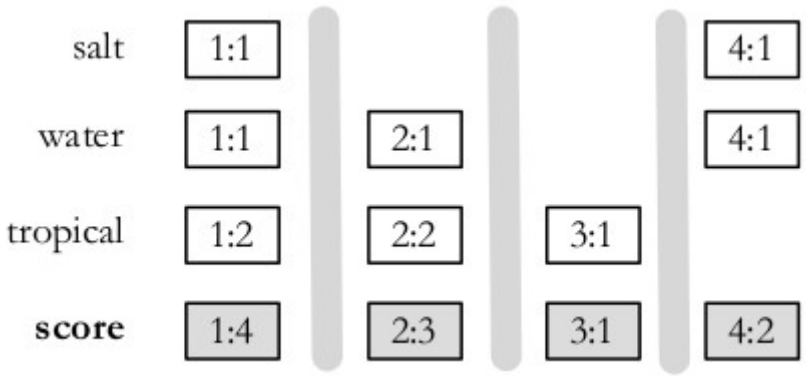
- Result merging
  - Index merging is a reasonable update strategy when index updates come in large batches
    - For single document updates, it isn't a very good strategy, since it is time-consuming to write the entire index to disk
  - For these small updates, it is better to just build a small index for the new data, but not merge it into the larger index
    - Could be in-memory, fast to update and search
    - Queries are evaluated separately against the small index and the big index, and the result lists are merged to find the top k results
  - Deletions are handled using delete list
    - Modifications done by putting old version on delete list, adding new version to new documents index

3.7 Query Processing

3.7.1 Document-at-a-time evaluation

- Calculates complete scores for documents by processing all term lists, one document at a time

Example query: "salt water tropical"



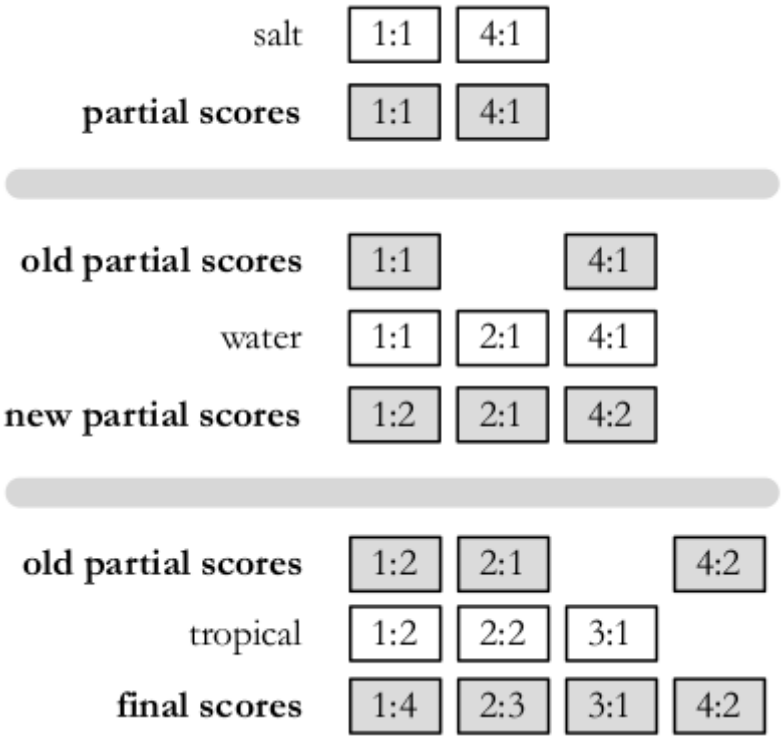
- The inverted lists are shown horizontally, although the postings have been aligned so that each column represents a different document
  - Inverted lists in this example hold word counts, and the score (assumed for this example, to be the sum of the word counts in each document)
  - Vertical gray lines indicate the different steps of retrieval. In the first step, all the counts for the first document are added to produce the score for that document

- Once the scoring for the first document has completed, the second document is scored, then the third, and then the fourth

3.7.2 Term-at-a-time evaluation

- Accumulates scores for documents by processing term lists one at a time

Example query: “salt water tropical”



- Both approaches have optimization techniques that significantly reduce time required to generate scores

3.7.3 Optimization Techniques

- See text for optimization techniques